

CoMem: Bounded-Read Long-Context Memory via Depth-Partitioned Computation Reuse

Anonymous ACL submission

Abstract

Retrieval bounds the number of online tokens but recomputes every model layer; KV reuse preserves computation but stores layer-wise states. We introduce **CoMem** (Comprehension Memory), which turns transformer depth into a second reuse axis. CoMem writes each context chunk once to an intermediate residual, retrieves a bounded subset for each query, and resumes only the upper layers with full cross-chunk attention. To our knowledge, it is the first long-context memory system to combine persistent independently written intermediate residuals, bounded query-time selection, and direct upper-layer replay. A lightweight self-distillation LoRA aligns this interface while the backbone remains frozen. At 128k, CoMem’s select-first online prefill is up to $64.9\times$ faster than dense full-context prefill; a Write-inclusive pipeline retains $2.74\times$ latency advantage at under 18.5 GB peak memory. A matched replay control exposes the depth-only quality–latency trade-off, while a continuous-state oracle exactly matches full replay, localizing the remaining fidelity gap to independent chunk writing and repositioning. CoMem therefore establishes persistent intermediate residuals as a practical long-context memory primitive rather than another token-axis cache.

1 Introduction

Long-context language models face two coupled scaling costs: processing more input tokens increases prefill latency, while retaining their layer-wise key–value (KV) states increases memory (Vaswani et al., 2017; Kwon et al., 2023). Both costs become acute when a large context must answer many queries.

Existing methods primarily reduce the *token* axis. KV compression retains fewer full-depth

states (Xiao et al., 2024b; Zhang et al., 2023; Li et al., 2024; Cai et al., 2024; Liu et al., 2024a), while retrieval-augmented systems select a bounded text or memory subset (Lewis et al., 2020; Xiao et al., 2024a; Tang et al., 2024; Xu et al., 2024). The former reuses model computation but materializes layer-wise state; the latter bounds online tokens but normally recomputes every layer. Neither treats transformer *depth* as a reusable memory axis.

Key observation. Task-relevant content can become accessible before a model’s native prediction pathway is complete. Across specified probes, linear accessibility precedes native LM-head readout; split sweeps likewise reveal a shallow readable region followed by a sharp depth boundary (§3). This does not locate “understanding” causally. It instead suggests a systems opportunity: persist an intermediate residual, then resume the remaining query-conditioned computation only for relevant chunks.

We introduce **CoMem** (Comprehension Memory), a bounded-read memory that makes depth an explicit reuse axis. During WRITE, independently processed chunks run through layers $[0:j)$ and persist their residuals. During SELECT, a lightweight selector retrieves a fixed number of chunks. During READ, those states are composed with the query and only layers $[j:L)$ execute with full cross-chunk attention. The split j spans a family: $j=0$ is matched raw-text full-depth replay, while larger j prepays more computation for repeated queries at a possible fidelity cost. A rank-32 self-distillation LoRA aligns the intermediate interface while leaving the backbone frozen.

To our knowledge, CoMem is the first long-context memory system to combine persistent independently written intermediate residuals, bounded query-time selection, and direct upper-layer replay. At 128k, its select-first online pre-

Anonymous repository: <https://anonymous.open.science/r/COMem-Anonymous/>

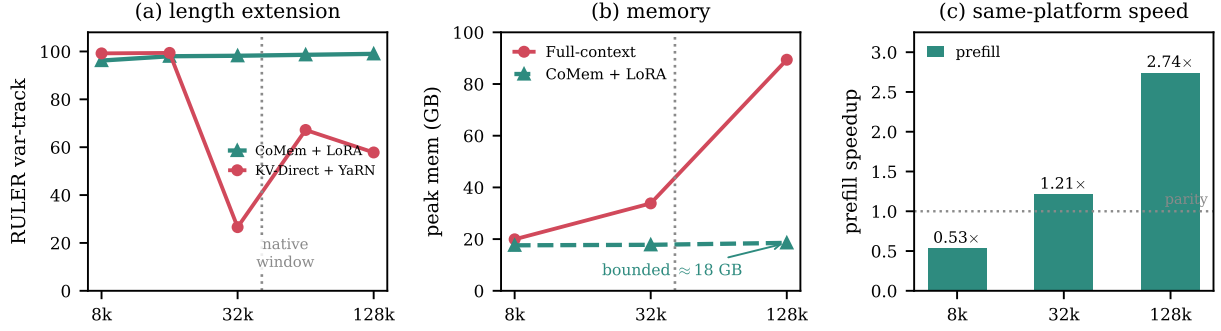


Figure 1: **CoMem turns bounded reads into a systems advantage.** On Qwen3-8B variable tracking, CoMem remains stable as context grows, while YaRN-enabled KV-Direct is length-sensitive. The same-platform pipeline cohort also shows bounded peak memory and increasing prefill advantage. Matched $j=0$ replay and same- j controls appear in §5.

fill is up to $64.9\times$ faster than dense full-context prefill after the reusable Write (external-store I/O excluded); a separate Write-inclusive comparison retains latency and memory advantages. Matched $j=0$, same-depth adaptation, and continuous-state oracle controls distinguish this system operating point from the smaller depth-only gain and localize the remaining fidelity gap to independent writing and repositioning.

Contributions.

- **A memory abstraction that** combines independently writable intermediate residuals, bounded content selection, and direct upper-layer replay; its split depth jointly controls storage, fidelity, and query-time work.
- **Depth as a measured reuse axis.** Cross-family probes, split sweeps, and self-distillation expose a tunable quality–cost frontier without updating the backbone.
- **Controlled systems evidence.** Long-context measurements show large bounded-read savings, while matched replay, oracle, adaptation, selector, and storage controls identify where the gains and losses arise.

2 Related Work

We organize prior work along the same three axes that structure our design (§3): *what* to cache, *how deep* to cache, and *how* to read it back. Table 1 summarizes the interface properties that define the resulting design space.

What to cache: full-depth KV vs. one intermediate residual tensor. The dominant approach to long context bounds the key–value (KV) cache

along the *token* axis, retaining a compressed subset of *full-depth* KV. StreamingLLM (Xiao et al., 2024b) keeps a recent window plus attention sinks; H2O/SnapKV (Zhang et al., 2023; Li et al., 2024) evict positions; PyramidKV, ZigZagKV, and LCKV (Cai et al., 2024; Zhong et al., 2025; Wu and Tu, 2024) vary budgets across layers; and MiniCache (Liu et al., 2024a) merges neighboring-layer KVs. CompressKV adds a retrieval-head criterion (Lin et al., 2026). Retained budget alone does not align quality and systems cost (Yuan et al., 2024). CoMem instead makes one depth- j residual tensor per chunk (one vector per token) the unit of a persistent, content-addressable memory, then reconstructs upper-layer computation only for the selected chunks.

Intermediate states for restoration or request-scoped compression. HCache (Gao et al., 2025) provides an important systems precedent for checkpointing intermediate activations, but its objective is to restore evicted execution state for a previously processed context. It does not formulate a memory that selects and composes independently stored chunks according to a new query. KV-Direct (Qasim et al., 2026) reconstructs full-depth KV from residuals but retains every token. ILRe (Liang et al., 2025) uses intermediate-layer keys for token retrieval within long-context compression, but does not persist independently written full residual tensors as cross-query memory objects or directly expose upper-layer replay as the memory depth knob. These works motivate intermediate-state reuse; CoMem is, to our knowledge, the first to turn it into bounded-read, depth-partitioned long-context memory.

Method family	Stored object	Cross-query persistence	Independent writes	Bounded query selection	Direct upper-layer continuation
Text RAG (Lewis et al., 2020)	token IDs	✓	✓	✓	—
Token-axis compression (Li et al., 2024; Cai et al., 2024)	full-depth KV	—	—	✓	—
HCache (Gao et al., 2025)	intermediate activation	context-bound	—	—	✓
KV-Direct (Qasim et al., 2026)	residual	request-bound	—	—	KV reconstruction
MemoryLLM (Wang et al., 2024)	latent pool	✓	—	fixed pool	—
CoMem	depth- j residual	✓	✓	✓	✓

Table 1: Interface-level comparison, not an empirical ranking. HCache is a checkpoint/replay precedent; MemoryLLM is our external persistent-memory reference. CoMem combines persistent independent residual writes, bounded query-conditioned selection, and direct upper-layer continuation.

How deep: fixed or trained memory vs. a tunable depth split. A second line fixes the memory by architecture or by training. MemoryLLM (Wang et al., 2024) is our principal external persistent-memory reference: it maintains a fixed-size latent pool self-updated inside the transformer. Its released Llama-3 checkpoint and constant-capacity memory differ from CoMem’s backbone and content-addressable residual store, so the comparison is informative but not matched. LongMem (Wang et al., 2023b) retrieves external states through a trained side network, and RecursiveSummarizing (Wang et al., 2023a) keeps a natural-language summary. YOCO (Sun et al., 2024) shares one global KV in a redesigned decoder–decoder, while CEPE (Yen et al., 2024) adds a parallel context encoder and cross-attention. Others induce compressibility through continued training: Activation Beacon (Zhang et al., 2024) learns to compress per-layer activations into beacon tokens, KV-CAT (Gelberg et al., 2026) trains for cache compressibility on the token/slot axis, and LLoCO (Tan et al., 2024) combines offline context compression with per-domain supervised LoRA. CoMem instead partitions on the *depth* axis and leaves the backbone frozen. Its inference-time memory operations require no parameter updates, while the evaluated flagship trains only a lightweight self-distilled LoRA; an adapter-free variant is reported separately. CoMem exposes j as a knob trading raw-text full-depth recomputation against deeper residual reuse (§3, *how deep*).

How to read: text RAG vs. residual retrieval. Conventional RAG recomputes all layers over retrieved raw text (Lewis et al., 2020); its trade-off with direct long-context processing has been studied explicitly (Xu et al., 2024). Our $j=0$ row is a matched raw-text replay control—same selector, top-12 IDs/order, sink, pack, and mask—rather than a representative RAG system. CoMem at $j=12$ selects the same chunks but retrieves

cached residuals and replays only upper layers. Landmark Attention (Mohtashami and Jaggi, 2023), FoT/LongLLaMA (Tworkowski et al., 2023), InfLLM (Xiao et al., 2024a), Quest (Tang et al., 2024), and RetrievalAttention (Liu et al., 2024b) and ACRE (Qian et al., 2025) use query-guided KV/activation retrieval. CacheBlend (Yao et al., 2024), RAGCache (Jin et al., 2024), and IMDC (Zeng et al., 2024) reuse or compress within-query states rather than CoMem’s persistent cross-query residuals.

3 Depth as a Design Axis

CoMem starts from a memory-specific observation: task-relevant information can be available before all query-conditioned computation is complete. We separate three pieces of evidence from the design principles they motivate. These are correlates under specified protocols, not a causal localization of understanding.

3.1 Linear accessibility precedes native readout

Under our probe protocol, semantic labels become linearly accessible near mid-depth, whereas the model’s own LM-head pathway reaches its knee later (Figure 2b). Upper states are also more query-conditioned (Lad et al., 2024). An intermediate state can therefore contain useful content without being directly readable by the native upper stack, motivating an explicit interface adapter rather than treating probe accuracy as generation readiness.

3.2 Readable depth has a boundary

Zero-shot recall stays high for shallow splits and then collapses when the cache moves deeper (Figure 2a). Self-distillation moves the usable point toward this boundary, but does not remove it. We therefore expose j as a quality–storage–Read knob instead of selecting one depth as an architectural constant. The flagship uses $j=12/36$; matched

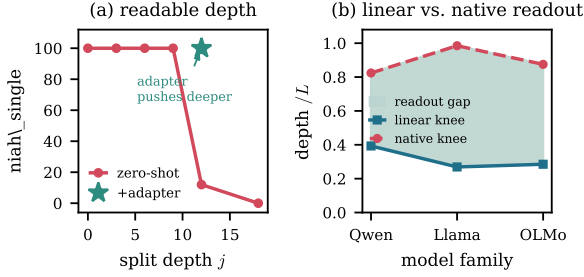


Figure 2: **Depth exposes a bounded, tunable cache point.** (a) Qwen3-8B zero-shot needle recall is stable at shallow splits and then collapses; a self-distilled adapter moves the readable point deeper. (b) Across the supported probe tasks, linear accessibility precedes native LM-head readout. Llama’s matched-support result is SST2-only because its WiC/RTE verbalizers stay near chance. Full protocols and results are in Appendix A.1.

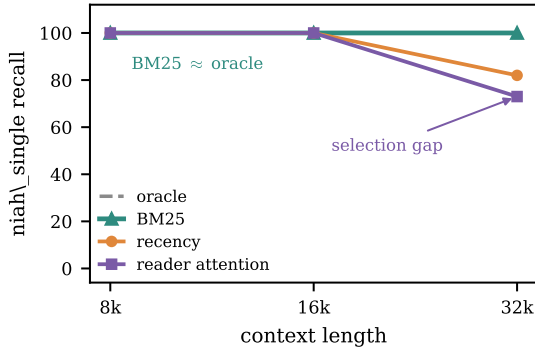


Figure 3: **Selection is the observed bottleneck on this diagnostic.** Oracle and BM25 remain perfect on the tested single-needle sweep, while recency and reader attention fall at 32k. Appendix Table 12 gives the full comparison and its limits.

$j=0$, frozen, and multi-depth controls appear in §5.

3.3 Selection bounds useful memory

Even a faithful cached state is useless when its evidence is absent from the read pack. On the lexical single-needle diagnostic, oracle and BM25 retain the gold evidence while recency and reader-attention selection degrade with length (Figure 3). This result is task-specific—it does not establish lexical retrieval as universally optimal—but it shows why a reusable residual store must be paired with bounded relevance selection.

3.4 Design implications

The evidence yields three principles. **First**, cache the complete per-token residual h_j , not a prediction-facing summary, and recompute the up-

per layers with the query present. **Second**, expose depth as a tunable operating point and adapt the native readout interface. **Third**, retrieve a bounded relevant pack and preserve full cross-chunk attention during upper-layer continuation. CoMem instantiates these principles as WRITE-SELECT-READ.

4 CoMem: Depth-Partitioned Computation Reuse

4.1 Overview and three read pathways

CoMem partitions a decoder along depth (Figure 4): Write caches one intermediate residual tensor per chunk; Select retrieves a bounded subset; Read recomputes the upper layers with the query present. Ranges are half-open, and h_j is the input to block j . The resumed computation is related to KV-Direct (Qasim et al., 2026), but adds shallow writes, a tunable split, and bounded external retrieval.

Three read pathways. Full context runs all L layers over every token. Text RAG bounds the prompt but still runs all layers. Our matched $j=0$ raw-text replay control fixes CoMem’s selector, pack, and mask while storing token IDs and replaying all layers. CoMem $j=12$ selects the same chunks, retrieves prewritten h_{12} , and runs only $[12:L)$. Both retrieved paths bound the token working set; only CoMem skips repeated lower-layer document computation, at the cost of residual storage and an approximate interface.

Notation and storage. Let the decoder have L layers, width d , head width d_{head} , split j , and chunk size c . CoMem stores $h_j \in \mathbb{R}^{c \times d}$. With a common dtype, its per-token storage relative to full KV is

$$\frac{|h_j|}{|\text{KV}|} = \frac{d}{2Ln_{\text{kv}}d_{\text{head}}} = \frac{n_q}{2Ln_{\text{kv}}}, \quad (1)$$

where n_q and n_{kv} are the query- and KV-head counts, and $d = n_q d_{\text{head}}$. Qwen3-8B uses $d=4096$, $L=36$, $n_q=32$, and $n_{\text{kv}}=8$, yielding 1/18 (8 KB vs. 144 KB per token in bf16). The residual store is about 1 GiB at 128k tokens and 8.2 GB at one million tokens: smaller than full KV, but roughly three orders of magnitude larger than token IDs or compressed text. Depth reuse is most attractive when this prepaid representation serves repeated queries. At read time we assemble a *pack*

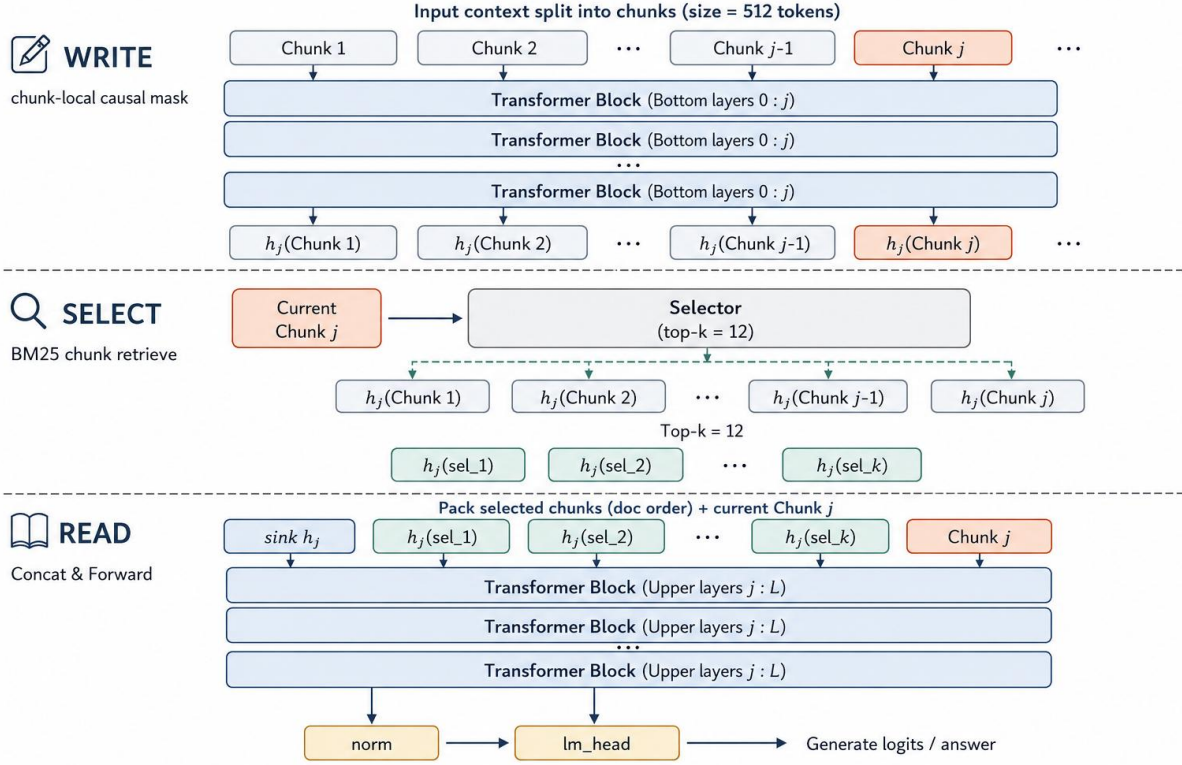


Figure 4: **CoMem pipeline.** WRITE encodes each 512-token chunk through $[0:j)$ and caches h_j . SELECT retrieves top- k relevant chunks. READ packs their hiddens with a sink and the query, then recomputes $[j:L)$ with full cross-chunk attention. The flagship reads approximately 6.5k tokens regardless of stored-context length.

whose length is fixed by k and c ,

$$|\text{pack}| = \text{sink} + k \cdot c + \text{query} \approx 6.7k, \quad (2)$$

for $k=12$, $c=512$, and—crucially—*independent of the context length*.

Why BM25? The flagship uses iterative token-ID BM25 as a deliberately lightweight selector: it is training-free, deterministic, CPU-only, and requires no embedding model or neural forward. It is also well matched to the entity and key overlap in our diagnostics. This choice isolates the memory interface from a separately trained retriever; CoMem itself is selector-agnostic, and we do not claim that lexical retrieval is universally optimal. Appendix A.1 compares BM25 with recency, reader-attention, and oracle selection and reports its scaling limits.

4.2 Write: caching intermediate residuals

The document is streamed in disjoint c -token chunks. Each chunk restarts rotary positions (Su et al., 2021) at zero, runs through $[0:j)$, and writes h_j to an external store keyed by its text. Chunk-local positions make cached states reusable regard-

Algorithm 1 CoMem: Write and Read

Require: backbone layers $[0:L)$, split j , chunk size c , top- k

- 1: **Write** (streaming, once per context):
- 2: **for** each c -token chunk x in the context **do**
- 3: $h_j \leftarrow \text{layers}_{[0:j)}(x)$ {chunk-local RoPE}
- 4: store h_j keyed by the chunk text
- 5: **end for**
- 6: **Read** (per query q):
- 7: $S \leftarrow \text{LEXICAL-TOP-}k(q)$ {bounded top-12 flagship retrieval}
- 8: $\text{pack} \leftarrow [\text{sink}; \{h_j\}_{s \in S}; h_j(q)]$ {fresh contiguous RoPE}
- 9: $\text{logits} \leftarrow \text{layers}_{[j:L)}(\text{pack})$ {full cross-pack attention}
- 10: **return** next-token logits

less of their original document offset. The Write cost is linear in context length and executes only j layers. The hidden store and BM25 index also remain linear; only the model-side online Read is bounded.

4.3 Depth split and self-distillation

The j knob. Split depth controls prepaid Write work versus per-query Read work. At $j=0$, the matched raw-text replay control performs full re-computation and matches the stock forward within $\max |\Delta_{\text{logit}}| < 10^{-4}$ on that same pack; at $j=L$, no upper layer is replayed. Intermediate j trades cheaper Read for query-blind readout loss. Self-distillation enables the flagship operating point $j=12/36$. KV-Direct is the separate no-retrieval full-context arm.

Self-distillation. Following knowledge distillation and low-rank adaptation (Hinton et al., 2015; Hu et al., 2022), we train a student at $j=12$ to match a frozen $j=0$ teacher on the teacher’s top-64 logit support. Let p and q be their renormalized distributions on that support. The actual objective is the bidirectional KL

$$\mathcal{L}_{\text{distill}} = 0.6 \text{KL}(p||q) + 0.4 \text{KL}(q||p), \quad (3)$$

with no auxiliary cross-entropy term. The canonical adapter trains rank-32 LoRA ($\alpha=64$, zero dropout) on all attention and MLP projections in layers 12–35 of a frozen Qwen3-8B. Training uses 4,000 steps of 8-GPU bf16 DDP (effective batch 8), AdamW with peak learning rate 8×10^{-5} , 100 warmup steps and cosine decay, on 2,048-token windows from the PG-19 training split (Rae et al., 2019). No synthetic long-context examples, task supervision, retrieval, or CE loss is used. This pushes the usable cache depth deeper and transfers without benchmark-specific supervision across the evaluated tasks (§5).

The backbone remains frozen. The $j=12$ flagship uses this LoRA, while a shallower adapter-free arm isolates the inference architecture. Designing compressible cache points during pretraining is complementary and outside our scope.

4.4 Read: bounded retrieval and recompute

The read pack. A selector returns top- k chunk indices. Their hiddens are ordered by document position and packed as $[\text{sink}; \text{selected } h_j; \text{query } h_j]$ with fresh, contiguous RoPE. Layers $[j:L]$ then run with full causal attention, allowing later chunks and the query to integrate earlier evidence. For fixed k and c , model-side Read FLOPs and KV working memory are independent of stored-context length; Table 14 tests the full-attention choice.

The sink is the backbone’s BOS token passed through layers $[0:j]$ as a one-token chunk. For generation, the query is first written with a lower-band KV cache, while the packed sink, selected states, and query are prefetched once through $[j:L]$ to create an upper-band cache. Each generated token then runs through $[0:j]$ at the next query-local position using the lower cache, and its result runs through $[j:L]$ at the next packed position using the upper cache; both position counters advance by one. The retrieved pack is never rerun per token. Decode work is therefore independent of stored-context length for a fixed pack and generated prefix. Table 24 verifies this path and reports timing.

Bounded lexical retrieval. BM25 (Robertson and Zaragoza, 2009) returns a bounded top- k set. The flagship uses $k=12$, hop width four, and three automatic frontier-expansion rounds with deduplication. Oracle, recency, reader-attention, and wider retrieval are analysis-only controls reported in §5 and Appendix A.1.

5 Experiments

5.1 Experimental setup

Models and protocol. Unless noted, the backbone is the released Qwen/Qwen3-8B checkpoint (Yang et al., 2025) (36 layers; revision b968826d9c46; full hash in Appendix A.4). We do no continued pretraining: the backbone remains frozen and the flagship’s only learned weights are a self-distilled rank-32 LoRA at $j=12$. All methods use plain-text prompting with `chat_template=False`, chunk size 512, a BOS sink, and iterative BM25. The read pack fixes top-12 and is about 6.5k tokens. Qwen3-8B’s native window is 40,960 tokens; 128k comparisons use YaRN (Peng et al., 2023) where stated.

Comparisons. Same-backbone systems include KV-Direct (Qasim et al., 2026), StreamingLLM (Xiao et al., 2024b), and InfLLM (Xiao et al., 2024a). Matched $j=0$ replay and same- j frozen/adapted controls isolate CoMem’s components. MemoryLLM (Wang et al., 2024) is the principal external persistent-memory reference, using its released Llama-3-8B-chat checkpoint (Grattafiori et al., 2024); its different backbone and fixed pool preclude matched attribution.

Benchmarks and statistics. We evaluate three named RULER families (Hsieh et al., 2024),

BABILong qa1/qa2/qa5 (Kuratov et al., 2024), six LongBench QA datasets (Bai et al., 2023), LongEval (Li et al., 2023), and LoCoMo (Maharana et al., 2024). RULER cohorts A and B differ in their single-needle variant and sampled examples; we label rather than pool them. Official task metrics are used throughout. LoCoMo’s semantic Judge is primary, with paired item and conversation-cluster bootstrap intervals and an independent-judge audit. Appendix B gives complete metrics, cohorts, aggregation, and integrity checks.

Hardware. Headline pipeline timing uses one NVIDIA L20A with the flagship LoRA enabled; the split-depth sweep uses one H20 and is compared only within its cohort. We report medians after warmup and separate online prefill, Write-inclusive pipeline, depth-only Read, decode, and external-store costs.

5.2 Main results

Table 2 separates same-backbone systems, the external MemoryLLM reference, and matched $j=0$ full-depth replay. CoMem trades quality relative to $j=0$ for skipping the lower 12 layers on repeated reads; it does not claim uniform accuracy dominance. On the matched length-extension comparison (Table 3), CoMem remains stable through 128k on the selected RULER tasks, whereas YaRN-enabled full-context behavior is task dependent. BABILong exposes a compression tax on qa1/qa2, while qa5 favors the bounded retrieval pathway.

Long-dialogue memory. Under the unified no-chat protocol, CoMem exceeds KV-Direct on LoCoMo. The ordering is supported by both item- and conversation-cluster resampling and an independent DeepSeek-V3 audit. Matched $j=0$ performs best, confirming that retrieval and depth reuse contribute separately.

5.3 Controlled comparisons

Equal retained-token budget. At the same nominal pack budget, CoMem outperforms StreamingLLM and the standard SnapKV/PyramidKV configurations on the native cohort (Appendix Tables 13 and 25). With YaRN and full prompts, the token-compression baselines recover quality but must first prefill the full 128k context. Thus retained tokens alone do not

imply matched online computation; saved method cohorts are not example-paired.

Depth and interface adaptation. Frozen $j=12$ is poor, while the same-depth distilled adapter recovers most of the selected-pack performance (Table 10), showing that linear accessibility is not native readability. A multi-depth sweep traces a monotonic quality–Read frontier until a $j=18$ boundary (Table 8). Because LoRA spans vary with j , this is a deployment frontier rather than a training-compute-matched causal ablation.

Where does the residual gap arise? On paired RULER-B, a continuous-prefix h_{12} oracle exactly matches full replay, whereas deployable chunk-local writing is lower (Table 27). The upper-layer continuation is therefore faithful when supplied continuous states; the observed deployable gap lies in the bundled effects of independent writing and repositioning. Cross-chunk and selector controls in Appendix A.1 show that full upper-layer interaction and relevant evidence selection are both necessary.

5.4 Efficiency and scaling

After a context has been written, CoMem’s fixed selected pack keeps online prefill near constant as the store grows; at 128k it is $64.9\times$ faster than dense full-context prefill (Table 5). This operating point combines bounded selection and depth reuse and excludes reusable Write and external-store fetch.

A separate same-adapter pipeline cohort includes document Write, CPU selection, query Write, and upper-layer Read; it retains a $2.74\times$ advantage at 128k and substantially lower peak GPU memory (Appendix Table 24). One-time index construction and external-store I/O remain separate. In the paired same-pack control, depth reuse alone provides $1.403\times$ model-side Read speedup at a 3.12-point RULER cost. Store lookup and I/O grow with corpus size even though model-side Read remains bounded; Appendix Tables 20 and 28 report the measured tiers.

5.5 Generality

The linear/native readout gap appears under the supported Qwen, OLMo, and Llama probe tasks, with the stated verbalizer limitations. Appendices additionally report model scaling, three adapter seeds, and two sparse MoEs, including bounded Hunyuan reads through 256k. These results test

Method	RULER [†] 15-cell	LongEval [†] 8k-128k	LongBench [†] 6-QA F1	BABILong [†] qa1/2/5 macro	LoCoMo Judge [†] full set
CoMem + LoRA ($j=12$)	97.05^A / 96.07 ^B	69.0	12.15	50.43	38.27
KV-Direct	78.80 ^A	65.2	12.17	63.00	34.59
InfLLM	77.83 ^A	21.6	11.86	59.52	22.21
StreamingLLM	23.37 ^A	30.8	11.11	48.14	25.63
MemoryLLM [†]	16.55 ^A	13.6	9.01	30.00	16.11
<i>Selected-pack full-depth replay reference (excluded from ranking)</i>					
Matched raw-text replay ($j=0$)	99.20 ^B	97.2	12.31	56.14	41.59

Table 2: Cross-benchmark results under the unified no-chat protocol. Bold marks the best same-backbone point estimate; RULER ranking uses cohort *A* only. [†]MemoryLLM uses a released 7B backbone with a fixed latent pool; it is listed as an external persistent-memory reference and is not budget-matched. The $j=0$ row retrieves the same raw-text chunks and recomputes all layers, providing the matched full-depth endpoint of CoMem’s depth axis. RULER superscripts denote distinct $n=100$ cohorts, which are never pooled. Table 10 isolates interface adaptation; Table 3 gives the length-extension comparison.

Task	Method	8k	16k	32k	64k	128k
single needle	KV-Direct	100	100	100	100	0
	KV-Direct+YaRN [†]	100	100	100	100	100
	CoMem+LoRA	100	91	97	98	98
	CoMem+LoRA+YaRN [†]	100	90	97	97	98
multikey	KV-Direct	100	100	98	88	0
	KV-Direct+YaRN [†]	98	94	96	91	89
	CoMem+LoRA	94	91	99	90	93
	CoMem+LoRA+YaRN [†]	82	85	94	88	93
variable tracking	KV-Direct	100	100	99.8	96.2	0
	KV-Direct+YaRN [†]	99.2	99.4	26.6	67.2	57.8
	CoMem+LoRA	96.2	98.0	98.2	98.6	99.0
	CoMem+LoRA+YaRN [†]	88.6	90.0	89.4	95.0	93.6

Table 3: Matched- $n=100$ Qwen3-8B RULER scaling. [†]YaRN factor 4 gives an effective 163,840-token window. At 128k, unextended-backbone CoMem+LoRA scores 98/93/99.0 on single-needle, multikey, and variable tracking, versus 100/89/57.8 for KV-Direct+YaRN. Applying YaRN to CoMem changes these to 98/93/93.6, showing an in-window position-scaling tax on variable tracking.

portability of the interface but do not establish a universal depth boundary.

6 Limitations and Discussion

CoMem is a bounded-read efficiency method, not an accuracy state of the art for every task. BABI-Long qa2 remains weak (27.0), and the matched $j=0$ raw-text replay control is a full-depth reference under the fixed pack, not a claim about every RAG implementation or a global quality upper bound. Results above Qwen3-8B’s native 40,960-token window are stress tests, not native-128k evidence. A 13-gram containment audit finds substantial PG-19 overlap in the InfiniteBench long-book support: a conservative 0.8 cut flags 24/86 books and 163/580 records, while the bimodal distribution gives 54/86 and 387/580 for any cut in 0.20–0.60. Because strict-clean-subset predictions were unavailable, we remove that quality comparison rather than risk contamination. RULER Cohorts

A/B differ in single-needle variant and sampled examples; we label them and never use a cross-cohort difference, but this reduces the number of methods available in the fully matched YaRN comparison.

The mechanism evidence is correlational: linear probes on SST2/WiC/RTE and fixed-verbalizer native readout motivate a reusable mid-depth band but do not causally locate understanding or establish the same knees on long-memory tasks. The fixed top-12 budget imposes a hard evidence ceiling: recall follows $\min(1, 12/E)$ once required evidence exceeds the budget. Co-keyed multi-value evidence is difficult for lexical BM25 even below that ceiling, global statistics fail, and five-link variable tracking remains weak despite perfect retrieval. Learned selectors, adaptive budgets, dynamic updates, deletions, long-form generation, code, multilingual, and multimodal memory remain open. All evaluated data are English;

Method	Judge	Judge _{1:4}	F1	Substr. acc.
CoMem + LoRA ($j=12$)	38.27	48.64	9.15	23.36
KV-Direct	34.59	43.83	9.02	22.36
StreamingLLM	25.63	31.04	7.67	13.75
InfLLM	22.21	26.43	7.39	13.34
MemoryLLM [†]	16.11	20.65	5.91	9.52

Table 4: LoCoMo comparison on all 1,986 chat-template-free examples. Judge_{1:4} is GPT-4o accuracy on the 1,540 answerable items; the full Judge column folds in local abstention scoring for 446 adversarial items. F1 and substring accuracy are deterministic lexical diagnostics; semantic Judge is the primary metric. All rows use the same scoring protocol. The same- j adapter effect is isolated in Table 7. On paired answerable items, distilled CoMem exceeds KV-Direct by 4.81 points (item-bootstrap 95% CI [2.34, 7.34]); a 10-conversation cluster bootstrap gives [1.27, 8.32] and 8/10 observed conversation differences favor CoMem. An independent DeepSeek-V3 audit preserves the ordering. [†]MemoryLLM is the external persistent-memory reference and uses its official chat-tuned Llama-3 checkpoint; its different backbone precludes matched attribution. Its native chat template does not improve this result.

Length	Dense prefill	CoMem prefill	Speedup	Dense peak	CoMem peak
8k	1.20 s	1.05 s	1.14×	18.8 GB	18.7 GB
32k	7.33 s	1.06 s	6.92×	25.0 GB	18.7 GB
128k	71.37 s	1.10 s	64.9×	50.0 GB	18.7 GB

Table 5: Select-first online prefill on one H20 (Qwen3-8B, bf16/SDPA, median of three after one warmup). Dense runs the stock full-context path; CoMem uses its rank-32 LoRA and a fixed 6,657-token top-12 pack. This cohort times online model prefill after the persistent memory is built; it excludes one-time document Write and external-store fetch. The speedup is therefore an online bounded-read operating point, not a production end-to-end or depth-only causal effect.

token-ID BM25 may transfer poorly to morphologically rich or non-whitespace-segmented languages with lower lexical overlap. The rank-32 LoRA is specific to CoMem’s split interface and is not training-budget matched to every inference-only baseline; frozen and $j=0$ controls expose but do not eliminate this asymmetry. MemoryLLM is the principal external persistent-memory reference, but its released backbone and fixed-capacity pool differ from CoMem’s linearly growing residual store, so we do not use it for causal or storage-matched claims. The adapted-HCache experiment is an interface diagnostic, not a head-to-head long-term-memory comparison. Chunk-local resets are

necessary for query- and pack-independent cached chunks, so their bundled cost is the deployable system’s real cost. A non-deployable continuous-prefix oracle now isolates this bundle: it matches full replay exactly, while the deployable gap arises from independent chunk writing/repositioning under the tested packs; it does not further separate position reset from missing lower-layer cross-chunk attention. SnapKV/PyramidKV add standard token-axis baselines at the same retained budget, but they first prefill the full prompt whereas CoMem reads from a persistent prewritten store. We therefore report full-prefill cost and avoid claiming a strictly compute-matched systems comparison. The multi-depth curve holds data order, seed, steps, optimizer, and loss fixed, but LoRA spans vary with j (72.745M to 43.647M parameters); it is a deployment frontier, not a training-compute-matched causal depth ablation. Cross-depth LoCoMo differences are point estimates without conversation-cluster intervals.

The storage benchmark measures deterministic bf16 tensors and random top-12 fetches, separately from BM25 and model inference. Its local NVMe and CEPH numbers are specific to one H20 node, filesystem, concurrency sweep, and O_DIRECT configuration. The 64.9×

online-prefill cohort assumes that persistent memory has already been written and excludes external-store fetch; its dense arm is the stock model whereas CoMem carries its interface LoRA. The separate 2.74×

pipeline timing attaches the same LoRA to both paths and includes document Write but excludes one-time BM25 index construction and external-store I/O. Neither number is production-stack end-to-end latency. Kernel choice, batching, hardware, and network topology can change absolute latency. The paired 1.403×

result applies only to the model read phase and excludes retrieval, reusable store/query Write, and persistent I/O; total decode latency at 16k is similar. Layer-12 replay is not quality preserving: it loses 3.12 RULER macro points, concentrated in distractor-heavy multikey retrieval, and the two arms construct layer-12 hidden states differently. We do not measure index-build energy or end-to-end datacenter cost.

LoCoMo contains only 10 conversation clusters. Cluster bootstrap and an independent-judge audit do not replace evaluation on more conversations, and the GPT-4o endpoint lacks a dated snapshot. The two additional LoRA seeds use ef-

fective batch 3 versus 8 for the flagship despite a matched data budget. Other ablations remain point estimates, and the 32B scale row uses $n=25$ rather than 500. Uniform no-chat prompting strengthens within-study comparability but limits external validity for chat- or instruction-formatted deployment. The available chat-template result is a sensitivity check, not a chat-matched replication of all headline benchmarks or adapter training.

7 Conclusion

CoMem introduces a new long-context memory primitive: persistent intermediate residuals that are independently written, selected under a bounded budget, and read by directly resuming upper-layer computation. To our knowledge, it is the first system to combine these three properties. At 128k, CoMem delivers up to $64.9\times$ online-prefill speedup, while a separate Write-inclusive comparison retains clear latency and memory advantages. Matched replay identifies the depth-only quality-latency frontier, and a continuous-state oracle shows that upper-layer continuation itself is lossless: the measured fidelity gap comes from independent chunk writing and repositioning. CoMem therefore establishes transformer depth as a practical reuse axis and isolates the key interface problem for future long-context memory systems.

8 Ethical Considerations

CoMem changes the cost and storage organization of an underlying language model; it does not add a safety mechanism. It therefore inherits risks such as hallucination, biased generation, unsafe completion, and unintended disclosure of sensitive text selected from a memory store. Lower-cost access to long histories could also scale benign applications and deliberate misuse alike. Cached residual tensors may also expose source information through inversion or membership-inference attacks; they should receive protections at least as strong as the source text. Their large persistent volume and possible cross-tenant reuse make authorization, isolation, and deletion operational requirements rather than properties conferred by non-readable tensor storage. Deployments should encrypt and audit the persistent store, enforce access control, filter or redact sensitive sources before Write, support verifiable deletion, restrict retrieval to authorized material, and retain application-appropriate output monitoring and hu-

man review. Although bounded reads reduce accelerator memory and per-query computation, training and large-scale evaluation still consume energy; we report the measured hardware and final training budget in Appendix A.4.

We collect no new human-subject data and recruit no annotators. Our experiments use previously released model, corpus, and benchmark artifacts; LoCoMo’s released conversations are generated by paired language-model agents rather than newly collected private conversations. The submitted software archive contains code only and excludes model weights, benchmark text, predictions, credentials, and API responses.

References

- Yushi Bai et al. 2023. LongBench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*.
- Zefan Cai et al. 2024. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*.
- Shiwei Gao, Youmin Chen, and Jiwu Shu. 2025. Fast state restoration in LLM serving with HCache. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*.
- Yoav Gelberg, Yam Eitan, Michael Bronstein, Yarin Gal, and Haggai Maron. 2026. Training transformers for KV cache compressibility. *arXiv preprint arXiv:2605.05971*.
- Aaron Grattafiori et al. 2024. [The Llama 3 herd of models](#). *arXiv preprint arXiv:2407.21783*.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- Cheng-Ping Hsieh et al. 2024. RULER: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- Chao Jin et al. 2024. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*.
- Yuri Kuratov et al. 2024. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *NeurIPS Datasets and Benchmarks*.

714	Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In <i>Proceedings of the 29th Symposium on Operating Systems Principles</i> .	770
715		771
716		772
717		773
718		774
719		
720	Vedang Lad, Wes Gurnee, and Max Tegmark. 2024. The remarkable robustness of LLMs: Stages of inference? <i>arXiv preprint arXiv:2406.19384</i> .	775
721		776
722		777
723	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuettler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In <i>Advances in Neural Information Processing Systems</i> , volume 33.	778
724		
725		779
726		780
727		781
728		782
729		
730	Dacheng Li, Rulin Shao, Anze Xie, Ying Sheng, Lianmin Zheng, Joseph E. Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. 2023. How long can open-source LLMs truly promise on context length? LMSYS Org Blog.	783
731		784
732		785
733		786
734		
735	Yuhong Li et al. 2024. SnapKV: LLM knows what you are looking for before generation. <i>arXiv preprint arXiv:2404.14469</i> .	787
736		788
737		789
738		790
739	Manlai Liang, Mandi Liu, Jiangzhou Ji, Huaijun Li, Haobo Yang, Yaohan He, and Jinlong Li. 2025. ILRe: Intermediate layer retrieval for context compression in causal language models. <i>arXiv preprint arXiv:2508.17892</i> .	791
740		792
741		793
742		
743	Xiaolin Lin, Jingcun Wang, Olga Kondrateva, Yiyu Shi, Bing Li, and Grace Li Zhang. 2026. CompressKV: Semantic-retrieval-guided KV-cache compression for resource-efficient long-context LLM inference. <i>arXiv preprint arXiv:2606.24467</i> .	794
744		795
745		796
746		797
747		
748	Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. 2024a. MiniCache: KV cache compression in depth dimension for large language models. <i>arXiv preprint arXiv:2405.14366</i> .	798
749		799
750		800
751		801
752	Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. 2024b. RetrievalAttention: Accelerating long-context LLM inference via vector retrieval. <i>arXiv preprint arXiv:2409.10516</i> .	802
753		
754		803
755		804
756		805
757		
758		806
759	Adyasha Maharana et al. 2024. Evaluating very long-term conversational memory of LLM agents. <i>arXiv preprint arXiv:2402.17753</i> .	807
760		808
761		809
762		810
763	Amirkeivan Mohtashami and Martin Jaggi. 2023. Landmark attention: Random-access infinite context length for transformers. <i>arXiv preprint arXiv:2305.16300</i> .	811
764		812
765		813
766		
767	Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2023. YaRN: Efficient context window extension of large language models. <i>arXiv preprint arXiv:2309.00071</i> .	814
768		815
769		816
		817
		818
	Kaleem Ullah Qasim, Jiashu Zhang, Muhammad Kafeel Shaheen, Razan Alharith, and Heying Zhang. 2026. The residual stream is all you need: On the redundancy of the KV cache in transformer inference. <i>arXiv preprint arXiv:2603.19664</i> .	819
		820
		821
	Hongjin Qian, Zheng Liu, Peitian Zhang, Zhicheng Dou, and Defu Lian. 2025. Boosting long-context information seeking via query-guided activation re-filling . In <i>Proceedings of ACL</i> , pages 9453–9464.	
	Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. 2019. Compressive transformers for long-range sequence modelling. <i>arXiv preprint arXiv:1911.05507</i> .	
	Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond . <i>Foundations and Trends in Information Retrieval</i> , 3(4):333–389.	
	Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced transformer with rotary position embedding. <i>arXiv preprint arXiv:2104.09864</i> .	
	Yutao Sun et al. 2024. You only cache once: Decoder-decoder architectures for language models. <i>arXiv preprint arXiv:2405.05254</i> .	
	Sijun Tan, Xiuyu Li, Shishir Patil, Ziyang Wu, Tianjun Zhang, Kurt Keutzer, Joseph E. Gonzalez, and Raluca Ada Popa. 2024. LLoCO: Learning long contexts offline. <i>arXiv preprint arXiv:2404.07979</i> .	
	Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-aware sparsity for efficient long-context LLM inference. In <i>International Conference on Machine Learning</i> .	
	Szymon Tworowski et al. 2023. Focused transformer: Contrastive training for context scaling. <i>arXiv preprint arXiv:2307.03170</i> .	
	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In <i>Advances in Neural Information Processing Systems</i> , volume 30, pages 5998–6008.	
	Qingyue Wang et al. 2023a. Recursively summarizing enables long-term dialogue memory in large language models. <i>arXiv preprint arXiv:2308.15022</i> .	
	Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. 2023b. Augmenting language models with long-term memory. In <i>Advances in Neural Information Processing Systems</i> .	
	Yu Wang et al. 2024. MemoryLLM: Towards self-updatable large language models. <i>arXiv preprint arXiv:2402.04624</i> .	

- Haoyi Wu and Kewei Tu. 2024. Layer-condensed KV cache for efficient inference of large language models. *arXiv preprint arXiv:2405.10637*.
- Chaojun Xiao et al. 2024a. InfLLM: Training-free long-context extrapolation with an efficient context memory. *arXiv preprint arXiv:2402.04617*.
- Guangxuan Xiao et al. 2024b. Efficient streaming language models with attention sinks. In *ICLR*.
- Peng Xu, Wei Ping, Xianchao Wu, Lawrence McAfee, Chen Zhu, Zihan Liu, Sandeep Subramanian, Evelina Bakhturina, Mohammad Shoeybi, and Bryan Catanzaro. 2024. Retrieval meets long context large language models. In *International Conference on Learning Representations*.
- An Yang et al. 2025. [Qwen3 technical report](#). *arXiv preprint arXiv:2505.09388*.
- Jiayi Yao et al. 2024. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*.
- Howard Yen, Tianyu Gao, and Danqi Chen. 2024. Long-context language modeling with parallel context encoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- Jiayi Yuan, Hongyi Liu, Shaochen Zhong, Yu-Neng Chuang, Songchen Li, Guanchu Wang, Duy Le, Hongye Jin, Vipin Chaudhary, Zhaozhuo Xu, Zirui Liu, and Xia Hu. 2024. [KV cache compression, but what must we give in return? a comprehensive benchmark of long context capable approaches](#). In *Findings of EMNLP*, pages 4623–4648.
- Zhiyuan Zeng, Qipeng Guo, Xiaoran Liu, Zhangyue Yin, Wentao Shu, Mianqiu Huang, Bo Wang, Yunhua Zhou, Linlin Li, Qun Liu, and Xipeng Qiu. 2024. [Memorize step by step: Efficient long-context pre-filling with incremental memory and decremental chunk](#). In *Proceedings of EMNLP*, pages 21021–21034.
- Peitian Zhang et al. 2024. Long context compression with activation beacon. *arXiv preprint arXiv:2401.03462*.
- Zhenyu Zhang et al. 2023. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*.
- Meizhi Zhong, Xikai Liu, Chen Zhang, Yikun Lei, Yan Gao, Yao Hu, Kehai Chen, and Min Zhang. 2025. [ZigZagKV: Dynamic KV cache compression for long-context modeling based on layer uncertainty](#). In *Proceedings of COLING*, pages 8897–8907.

A Additional Results and Ablations

This appendix contains the detailed task tables, mechanism ablations, and cross-scale generality results summarized in the eight-page main paper. Appendix B separately documents aggregation, uncertainty, and the LoCoMo judge protocol.

Content-depth probe protocol. For SST2, WiC, and RTE we extract every layer once from a fixed stratified pool (up to 3,000 train examples), then fit L2 logistic regressions on five independent 60/20/20 train/dev/test splits. Features are standardized; $C \in \{0.1, 1, 10\}$ is selected on development data. For layer accuracy $a(l)$, the knee is $\min\{l : a(l) \geq 0.98 \max_l a(l)\}$. The native readout instead applies the model’s own final norm and LM head to each layer using fixed verbalizers. Lexical uni/bi-grams, token length, random labels, majority class, and balanced training serve as controls. On SST2, probe peak is about 0.90 versus 0.71 lexical, 0.56 position/majority, and 0.54 random-label accuracy; WiC/RTE are lower-selectivity and have wider per-task intervals. The reported confidence intervals are Student- t intervals over task-split knees, not training-seed uncertainty.

A.1 Mechanism ablations

Split depth and self-distillation. Table 6 reports linear and native-readout knees over five stratified data splits for Qwen and OLMo, plus a matched-support Llama SST2 result; protocol details follow below. Table 7 gives the frozen $j = 0/6/9/12$ quality sweep and same- j distilled control; Table 8 gives separately distilled $j = 6/9/12/18$ deployment points; Table 9 combines quality, storage, and same-H2O cost for the four core operating points. Table 10 provides a separate same- j RULER control. Finally, Table 11 uses an HCache-style checkpoint path only as an interface-adaptation diagnostic, showing that the readout repair transfers beyond CoMem.

Selection. Table 12 compares oracle, BM25, recency, and reader attention. BM25 is near oracle on the lexical needle tasks and is more reliable than the tested training-free alternatives at long lengths. Variable tracking instead requires iterative expansion along literal variable links; a three-round CPU microbenchmark costs 188.8ms at 128k, versus 41.0ms for one-shot BM25, without an additional model forward. This

Model	L	linear knee (/ L , 95% CI)	native knee (/ L)	gap (Δ/L)
Qwen3-8B	36	0.393 [0.319,0.466]	0.824	0.43
Llama-3-8B [†]	32	0.275	1.000	0.725
OLMo-2-7B	32	0.285 [0.214,0.357]	0.875	0.59

Table 6: **Linear accessibility precedes native readout under these probes.** The linear knee is the first layer reaching 98% of peak held-out probe accuracy; the native knee applies each model’s own final norm and LM head at every layer. Qwen and OLMo aggregate SST2, WiC, and RTE over five stratified data splits; their intervals reflect task-split variation, not model-training seeds. [†]For Llama, WiC and RTE native verbalizers remain near chance, so we report the SST2-only matched-support knees and do not form a misleading three-task native aggregate. These correlational probes do not localize understanding or establish the same knees on long-memory tasks.

Configuration	Split j	LoRA	LoCoMo Judge	BABILong mean
				qa1 / qa2 / qa5
Matched raw-text replay	0	no	41.59	66.3 / 38.9 / 63.3
CoMem frozen	6	no	32.78	44.6 / 22.6 / 53.9
CoMem frozen	9	no	29.15	42.4 / 19.6 / 55.6
CoMem frozen	12	no	24.52	33.4 / 18.0 / 60.3
CoMem distilled	12	yes	38.27	55.6 / 27.0 / 68.7

Table 7: Controlled comparison of retrieval, split depth, and self-distillation using Qwen3-8B. All configurations follow the same plain-text prompting protocol, with model-specific chat templating disabled (`chat_template=False`), and retrieve the same top-12 pack. For the frozen configurations, readout fidelity decreases as the split moves deeper. At the same split $j=12$, the rank-32 LoRA improves the LoCoMo Judge score by 13.75 points and the BABILong qa1, qa2, and qa5 means by 22.2, 9.0, and 8.4 points, respectively. BABILong entries are rounded task means from the underlying 21 task-length cells; the raw-cell $j=0$ macro is 56.14. The $j=0$ configuration performs full-depth recomputation over the retrieved pack, providing a selected-pack reference without lower-layer reuse and thus no depth-based reduction in query-time computation.

is a low-overhead selector in the tested regime, not a constant-time one: lookup grows approximately linearly with store size (Table 20). Dense or learned retrieval remains untested, so our results establish a transparent lexical operating point rather than selector optimality.

Read interaction and chunk size. Table 14 compares full cross-chunk attention with block-diagonal reuse. Table 15 reports the chunk-size sweep; its entries are point estimates rather than a multi-seed stability ranking.

Split j	LoRA params	RULER B	Full — deployed (95% CI)	LoCoMo	Read (ms)	Read speedup	Write (ms)
6	72.745M	98.29	0.91 [0.49, 1.39]	40.38	830.3	1.166×	133.4
9	65.470M	97.55	1.65 [1.04, 2.31]	39.02	748.3	1.273×	196.9
12	58.196M	96.07	3.12 [2.36, 3.93]	38.27	664.4	1.403×	—
18	43.647M	55.41	43.79 [41.77, 45.85]	28.65	499.5	1.807×	390.3

Table 8: Distilled depth curve. Each split has a separately trained rank-32 adapter on layers $[j:36]$ under the same data order, seed, steps, optimizer, and KL recipe. RULER uses Cohort B ($n=1,500$ paired examples); gap CIs are paired bootstrap intervals against full replay with that depth’s adapter, and all McNemar tests reject equality ($p \leq 3.05 \times 10^{-5}$). LoCoMo reports the full-set GPT-4o judge point estimate ($n=1,986$). Read/Write use a fixed 16k same-pack input and three process medians; retrieval, I/O, and decode are excluded, and total latency remains decode-dominated. The matched fixed-pack Write value was not retained for the earlier $j=12$ artifact. Adapter spans and parameter counts differ, so this is not a training-compute-matched depth ablation.

Operating point	RULER	B/token	Query (s)	Write (s)	Peak (GB)
Full context	78.80 ^A	147,456	50.59	0	50.0 [†]
Matched raw-text replay ($j=0$)	99.20^B	~4–8	1.141	0.09	18.5
CoMem frozen ($j=12$)	8.01 ^A	8,192	0.849	5.83	18.5
CoMem + LoRA ($j=12$)	96.07 ^B	8,192	0.807	5.83	18.5

Table 9: Representative quality and systems operating points, not a single matched Pareto frontier. Quality is a 15-cell RULER macro from the explicitly marked A/B cohort; systems columns are measured at 128k on one H20 (bf16/SDPA, median of three). Superscripts follow Table 2; the frozen quality point is not compared numerically across cohorts, while its direct same-task collapse appears in Table 10. All three retrieved systems rows use the same top-12 working set. The $j=0$ row stores raw token IDs and replays all layers; the $j=12$ rows store residuals and replay only the upper layers. The $j=0$ row has no LoRA whereas the flagship $j=12$ row is adapted; their 0.33 s/query difference is therefore a deployable operating-point comparison, not depth-only causal isolation. It prepays 5.74 s of Write and breaks even after roughly 17–20 repeated queries. Frozen $j=12$ shows that depth reuse requires interface adaptation. [†]Full-context KV-cached generation peaks at 50.0 GB; a separate single-forward write-all harness that materializes full-sequence logits OOMs on H20. The 89.39 GB L20A cohort in Table 24 is therefore not a contradictory hardware capacity claim.

Task	Adapter	8k	16k	32k
niah_single_2	off	36	7	22
	on	100	100	100
niah_multikey_1	off	8	3	1
	on	91	95	92

Table 10: Separate controlled RULER self-distillation ablation on Qwen3-8B at the same split $j=12$ ($n=100$ per cell). Each on/off pair uses the same task, length, selector, and cohort, so the difference isolates the adapter. Table 7 provides the broader same- j LoCoMo/BABILong control used in the main argument.

Configuration	LoRA	LoCoMo Judge
HCache-style ($j=12$)	no	13.29
HCache-style ($j=12$)	yes	31.17

Table 11: Interface-adaptation diagnostic on an HCache-style, retrieval-free checkpoint path. The two configurations share the $j=12$ split and evaluation harness, isolating the adapter. The gain shows that aligning intermediate states to native upper-layer readout transfers beyond CoMem; this is not a head-to-head long-term-memory comparison.

Task	Len	BM25	Recency	ReaderAttn	Oracle
niah_single	8k	100	100	100	100
	16k	100	100	100	100
	32k	100	82	73	100
niah_multikey	8k	99	98	97	100
	16k	99	88	90	100
	32k	99	54	60	100
var-track	8k	99.4	99.2	99.8	N/A
	16k	92.6	92.4	92.4	N/A
	32k	32.0	41.2	27.8	N/A

Table 12: Single-pass selector sweep on RULER ($n=100$) without a chat template; each entry is the peak over $k \in \{4, 8, 12, 16, 24\}$. Oracle retrieval keeps both needle tasks at 100, showing that their long-range gap is selection rather than readout loss. The variable-tracking rows belong to this specific selector-sweep configuration and should not be conflated with the focused top-12 flagship in Table 13. Oracle is undefined for a multi-chunk reference chain and is therefore not reported there.

Method	RULER task	8k	16k	32k	64k	128k
CoMem	niah_single_2	100.0	100.0	99.0	99.0	100.0
	var-track	96.6	97.6	98.8	99.0	95.8
StreamingLLM	niah_single	85.0	36.0	20.0	10.0	2.0
	var-track	41.8	2.2	0.6	0.2	0.8

Table 13: Equal-budget, no-chat RULER comparison ($n=100/\text{cell}$). CoMem and StreamingLLM use the same nominal budget and scorer but not paired examples; CoMem retrieves about 6.5k relevant tokens, whereas StreamingLLM keeps recent tokens.

Task	Full attn.	Block-diag.	Δ
RULER niah_single	100 / 100	100 / 96	0 / +4
RULER niah_multikey	96 / 94	60 / 32	+36 / +62
BABILong qa2	36 / 20	17 / 13	+19 / +7
BABILong qa5	78 / 69	78 / 55	0 / +14

Table 14: Full cross-chunk attention versus block-diagonal reuse (8k/16k, no chat). RULER uses $n=50$ and BABILong $n=100$ with identical top-12 retrieval. Joint recomputation matters most for multi-fact tasks.

chunk	read tokens	uncached step (s/token)	8k	16k	32k	64k
128	1,665	0.19	91.0	90.0	81.0	85.0
256	3,329	0.37	80.0	90.0	90.0	94.0
512	6,657	0.72	89.0	95.0	97.0	94.0
1024	13,313	1.60	100.0	89.0	92.0	94.0

Table 15: Chunk-size trade-off on RULER niah_multikey ($n=100$, no chat). Packed-step cost grows approximately linearly with read length; accuracy values are point estimates, not a multi-seed claim.

A.2 Full chat-template-free benchmark comparisons

Tables 16–19 give the full audited comparisons for RULER, LongEval, BABILong, and LongBench. Main rows use the unified no-chat protocol; MemoryLLM is the external persistent-memory reference using its released Llama-3 checkpoint, while the marked LLoCO row uses its released native setup. The all-method LoCoMo comparison is Table 4; judge details are in Appendix B.

Method	Task	8k	16k	32k	64k	128k
CoMem + LoRA	single-2	100	100	99	99	100
	multikey	95	94	97	91	93
	var-track	96.6	97.6	98.8	99.0	95.8
CoMem frozen ($j=9$)	single	96	99	99	99	96
	multikey	44	44	59	30	48
	var-track	45	38.8	36.4	31.2	25.8
KV-Direct	single	100	100	100	100	0
	multikey	100	100	98	88	0
	var-track	100	100	99.8	96.2	0
InfLLM	single	100	100	92	61	57
	multikey	100	79	65	45	20
	var-track	100	96.8	90.6	81.8	79.2
StreamingLLM	single	85	36	20	10	2
	multikey	83	34	18	13	4
	var-track	41.8	2.2	0.6	0.2	0.8
<i>External persistent-memory reference (different backbone)</i>						
MemoryLLM [†]	single	29	40	37	21	21
	multikey	28	24	25	12	10
	var-track	0	1.2	0	0	0

Table 16: Full chat-template-free RULER comparison using official `string_match_all` ($n=100$ per cell). KV-Direct places the full context in the read with native RoPE and no YaRN; its 64k/128k cells are unextended stress references rather than a length-extended upper bound. CoMem keeps a bounded retrieved pack. This is **Cohort A**: every method uses `niah_single_2`, `niah_multikey_1`, and `variable_tracking`, with $n=100$ and the official scorer. Its CoMem macro is 97.05. Cohort B instead uses `niah_single_3` to match the YaRN arm (Table 3) and yields 96.07; the cohorts are never pooled in a direct comparison. [†]MemoryLLM is the principal external persistent-memory reference and uses its released Llama-3-8B-chat checkpoint; its different backbone precludes matched attribution.

Method	8k	16k	32k	64k	128k	Mean
CoMem + LoRA	69	75	64	67	70	69.0
CoMem frozen ($j=9$)	8	0	4	0	4	3.2
KV-Direct	100	96	92	38	0	65.2
StreamingLLM	86	34	18	10	6	30.8
InfLLM	60	30	12	4	2	21.6
MemoryLLM [†]	22	22	16	6	2	13.6

Table 17: LongEval register-content accuracy. Unmarked rows use the unified chat-template-free protocol. Means use the common 8k–128k support. KV-Direct uses native RoPE without YaRN, so its 64k/128k cells are unextended stress references and the five-length mean is not a length-extended upper bound. CoMem’s separate six-length headline is 72.83 after including its 4k score of 92. The frozen CoMem control used 48 generation tokens rather than 16; both budgets exceed the single-number answer length. [†]MemoryLLM is the external persistent-memory reference and uses a different chat-tuned backbone, so its row is not a matched comparison.

Method	Task	0k	1k	2k	4k	8k	16k	32k	Mean
CoMem + LoRA	qa1	98	80	68	68	46	17	12	55.6
	qa2	26	44	43	44	23	8	1	27.0
	qa5	68	76	76	75	68	60	58	68.7
CoMem frozen ($j=9$)	qa1	98	59	71	57	7	1	4	42.4
	qa2	53	17	28	26	8	4	1	19.6
	qa5	70	74	65	60	41	39	40	55.6
KV-Direct	qa1	98	84	80	74	80	72	63	78.7
	qa2	58	53	50	46	49	49	37	48.9
	qa5	71	73	62	59	65	42	58	61.4
InfLLM	qa1	97	84	81	72	69	52	34	69.9
	qa2	51	58	41	47	41	39	30	43.9
	qa5	69	68	63	66	73	60	55	64.9
StreamingLLM	qa1	97	83	80	71	23	27	12	56.1
	qa2	49	56	44	49	19	11	4	33.1
	qa5	68	65	68	65	39	47	34	55.1
MemoryLLM [†]	qa1	52	46	35	28	23	17	12	30.4
	qa2	37	29	17	18	21	17	11	21.4
	qa5	50	45	39	35	35	34	29	38.1

Table 18: BABILong accuracy ($n=100$ per task-length cell), evaluated with the official `compare_answers` scorer and `TASK_LABELS`. Within the native context window, full-context KV-Direct is substantially stronger on qa1 and qa2, exposing the compression tax, whereas distilled CoMem achieves the highest qa5 mean, ahead of InfLLM (68.7 vs. 64.9). All methods are evaluated under the unified plain-text prompting protocol, with model-specific chat templating disabled (`chat_template=False`). [†]MemoryLLM is the principal external persistent-memory reference and uses its released Llama-3-8B-chat checkpoint. Its different backbone precludes matched attribution; the official chat template does not improve these diagnostics (Appendix A).

Method	NQA	Qasper	Hotpot	2Wiki	MultiF.	Musique	Macro
CoMem + LoRA	4.12	11.01	11.62	12.83	25.41	7.91	12.15
CoMem frozen ($j=9$)	4.63	11.23	9.41	10.71	22.05	5.72	10.63
KV-Direct	3.70	11.82	12.68	12.03	25.30	7.49	12.17
InfLLM	2.99	11.35	12.08	12.50	25.33	6.94	11.86
StreamingLLM	3.49	11.64	11.02	12.19	22.42	5.88	11.11
MemoryLLM [†]	3.13	8.46	8.76	10.27	17.43	5.98	9.01
LLoCO [‡]	24.21	24.45	44.24	—	—	—	30.97

Table 19: Official LongBench QA token F1 by dataset. Except for the explicitly marked LLoCO diagnostic, methods use the same plain-text no-chat protocol. CoMem with LoRA scores 12.15 and full-context KV-Direct scores 12.17; because only CoMem uses the split-interface adapter, this near-equality is descriptive rather than a training-budget-matched compression estimate. The frozen CoMem configuration reports adapter-free performance at $j=9$. NQA denotes NarrativeQA, and MultiF. denotes MultiFieldQA-en. [†]MemoryLLM is the principal external persistent-memory reference and uses its released Llama-3-8B-chat checkpoint. Its different backbone precludes matched attribution; its official chat template does not improve the tested diagnostics (Appendix A). [‡]LLoCO uses LLaMA-2-7B AutoCompressor, released per-domain supervised LoRAs, fp16, and its native task prompts. Released weights cover only NarrativeQA/Qasper/HotpotQA on our 200-example sets; 30.97 is their three-task macro and is not comparable to the six-task macro column.

A.3 Store scaling and failure boundaries

Table 20 tests the “bounded read over extensible store” claim directly. Retrieval is invariant when the required evidence fits top-12, but lookup and index construction still scale with store size. Beyond the budget, recall is physically capped; co-keyed multivalued evidence falls below even this cap (recall 0.62 at $E=8$), and a common-word-frequency task scores zero as coverage falls from 4.5% at 128k to 1.2% at 512k. These negative results delimit the claim: bounded retrieval is not global aggregation, and successful retrieval does not guarantee multi-hop reasoning.

We audited the PG-19 adapter corpus against the InfiniteBench long-book support using normalized 13-gram containment sketches. The conservative 0.8 threshold flags 24/86 books (163/580 records), but containment is sharply bimodal: 31 books are below 0.10, no book lies in 0.18–0.60, and 54 are at least 0.60. Any cut in 0.20–0.60 therefore flags 54/86 books and 387/580 records; anonymized character names make the 0.8 count an undercount. Existing predictions were unavailable for a strict-clean-subset rescore (113 QA and 76 choice records), so we remove the book-quality

Store	NIAH recall	NIAH score	VT recall	VT score	Read tokens	BM25 (ms)
128k	1.00	100	1.00	18	6,211 / 6,463	80 / 20
256k	1.00	100	1.00	34	6,209 / 6,463	164 / 39
512k	1.00	100	1.00	40	6,209 / 6,464	328 / 81
1M	1.00	100	1.00	54	6,209 / 6,463	697 / 170
2M	1.00	90	1.00	38	6,211 / 6,463	1,407 / 357
4M	1.00	100	1.00	34	6,208 / 6,463	2,852 / 725

Table 20: Store scaling with a fixed top-12 read. Values separated by “/” are single-evidence NIAH / five-link variable tracking (VT). Evidence recall ($n=20$) stays perfect and the model read stays about 6.2–6.5k tokens from 128k to 4M stored tokens; BM25 lookup grows approximately linearly. Generation scores use $n=10$ and reveal that VT reasoning remains noisy even when all five evidence chunks are retrieved. At 128k, increasing required VT evidence from 12 to 16/24/32 gives recall 1.0 to 0.75/0.50/0.375, exactly $\min(1, 12/E)$.

comparison rather than treat it as out-of-domain evidence. This does not affect the synthetic store-scaling results above.

A.4 Reproducibility details

Tables 21–23 record the configuration, training, and evaluation settings used by the Qwen3-8B flagship. The archived configuration files and evaluation scripts accompany the code release.

Artifacts, licenses, and data scope. The study evaluates English text only. It uses the frozen released Qwen/Qwen3-8B checkpoint (Apache-2.0); we do no backbone continued pretraining. It also uses PG-19 (Rae et al., 2019), RULER (Hsieh et al., 2024), BABILong (Kuratov et al., 2024), LongBench (Bai et al., 2023), LongEval/LongChat (Li et al., 2023), and LoCoMo (Maharana et al., 2024). RULER and LongChat code are Apache-2.0; the LongBench repository is MIT-licensed, while its component datasets retain their upstream terms; BABILong code is Apache-2.0 and incorporates BSD-licensed bAbI tasks; LoCoMo is CC BY-NC 4.0. PG-19 contains pre-1919 Project Gutenberg books, for which the dataset page does not assert one uniform corpus license, so we do not redistribute its text. We likewise redistribute no model weights, benchmark examples, predictions, or API responses. The anonymous code archive contains source, documentation, pinned environment requirements, and third-party notices; users obtain all external artifacts from their original providers and remain responsible for their terms. Table 23 reports the evaluated supports and sample counts; LoCoMo category denominators are in Appendix B.

Software environment. The archived environment specifies Python 3.10+, PyTorch 2.10, Trans-

formers 5.5.4, PEFT 0.10+, Datasets 2.14+, bf16, and SDPA. Evaluation uses the benchmark-specific official scorers named in Table 23; the archive records all non-default flags and provides a CPU correctness gate.

Compute accounting. The final adapter run used one node with 8 H20 GPUs for about 22 minutes, or approximately 2.9 H20 GPU-hours. Tables 22 and 24 report model size, trainable parameters, hardware, and measured timing protocol. Total GPU-hours across preliminary probes, failed runs, baseline generation, and all ablations were not consistently logged; we report this omission rather than extrapolating an unreliable total.

Efficiency measurement. Headline Table 24 uses one NVIDIA L20A with PyTorch 2.10, Transformers 5.5.4, bf16, SDPA, and the rank-32 LoRA enabled, reporting medians of three repetitions. Full-context prefill times the complete forward; CoMem times document Write, CPU retrieval, query Write, and upper-layer Read through logits. The timer excludes one-time BM25 index construction and external-store I/O; Table 28 measures the latter separately. At 128k, full-context prefill peaks at 89.39 GB while CoMem peaks at 18.54 GB. Cached decode uses 20 greedy steps.

The representative operating-point cohort (Table 9) is a separate one-H20 bf16/SDPA sweep; its A/B quality values are not a single matched Pareto frontier. Retrieved rows share top-12 IDs, a 6,657-token nominal pack (measured ~ 6.5 k), and chunk size 512. Write-all timing processes the complete store once; Read is per query. The $j=0$ text-retrieval path has no adapter, whereas $j=12$ uses the flagship LoRA; hence the 0.33 s Read difference is an operating-point comparison rather than a depth-only effect. Its 5.74 s extra Write implies a 17–20-query break-even. CoMem stores

Layer	Setting	Value
Backbone	Architecture	Frozen Qwen/Qwen3-8B; revision b968826d9c46 (full hash in artifact manifest); $L=36$, $d=4096$, 32 query heads, 8 KV heads, bf16, SDPA.
Backbone Write	Positions Cached state	Native window 40,960; RoPE $\theta=10^6$; no active RoPE scaling or YaRN. Split $j=12$; disjoint 512-token chunks; chunk-local positions restart at zero; one bf16 residual tensor h_j per token.
Read	Pack	[BOS; selected h_j ; $h_j(q)$] in document order, with fresh contiguous RoPE and full causal cross-chunk attention. The nominal cap is $1 + 12 \times 512 + 512 = 6,657$ tokens; actual RULER means are about 6.2–6.5k because tail chunks and queries can be shorter.
Retrieval	BM25	Token-ID documents and bare-question token IDs; $k_1=1.5$, $b=0.75$, Robertson IDF with +1 smoothing; no lowercasing, stemming, or stop-word removal.
Retrieval	Iteration	<code>rounds=0</code> means automatic $\lceil k/h \rceil$ rounds, not one-shot. Each round uses newly selected chunks as the next frontier and deduplicates the selected set.
Retrieval	Width	All five benchmarks fix $k=12$, hop width $h=4$, and therefore three automatic rounds. Chunk size 512 and the BOS sink are also fixed.
Read	Mask	The pack is one causal sequence: the final query sees all chunks; later chunks may attend to earlier chunks. In the block-diagonal ablation, chunks see only themselves and the sink, while the final query still sees all chunks.
Generation	Decoding	Greedy decoding (no sampling, temperature, top- p , or beams); no chat template and no thinking mode.
Adapter-free arm	Deployment	Frozen backbone, no LoRA, $j=9$; all other memory and evaluation settings are held fixed.

Table 21: Flagship CoMem inference configuration. The retrieval and read configuration is frozen across all five benchmarks; only benchmark harness parameters differ.

Setting	Value
Adapter	Rank 32, $\alpha=64$, dropout 0; Q/K/V/O and gate/up/down projections in layers 12–35 only; 58.20M trainable parameters (0.71%). Final weight SHA-256: dd09cd17457c63578c0f38dab79b287ab5da6e3f14c119aedefec1c34400536f.
Teacher and objective	The same Qwen3-8B with adapters disabled and $j=0$ serves as the frozen teacher. On the teacher top-64 support, $\mathcal{L} = 0.6 \text{KL}(p q) + 0.4 \text{KL}(q p)$ with implicit temperature 1 and no CE term. The loss is applied only to the query segment.
Data	Plain PG19 train text; no synthetic long-context examples, retrieval, task labels, or benchmark supervision. Training uses 2,048-token windows, formed as four 512-token chunks.
Optimization	AdamW, $\beta = (0.9, 0.95)$, weight decay 0, peak LR 8×10^{-5} , 100-step linear warmup, cosine decay to zero, gradient clipping 1.0, seed 42, bf16, no gradient checkpointing.
Schedule	4,000 steps on 8 GPUs, one sample per GPU and no gradient accumulation (global batch 8), totaling about 65.5M tokens.
Compute	One node with 8×NVIDIA H20 GPUs. Logged throughput is about 24.5 samples/s, implying roughly 22 minutes wall-clock; training peak memory was not recorded.

Table 22: Self-distillation and LoRA training configuration. The adapter name’s “4k” denotes 4,000 optimization steps; the training sequence length is 2,048 tokens.

Benchmark	Tasks and support	Generation	Scoring and aggregation
RULER	Single needle, multikey, and variable tracking; 8k–128k; 100 examples/cell; 8 shards; seed 42.	48 tokens; 60 for variable tracking.	Official case-insensitive <code>string_match_all</code> ; equal-weight macro over 15 cells.
LongEval	Lines retrieval; 4k–128k; 100 examples/length; 8 shards; seed 1234.	16 tokens for the flag-ship; 48 for frozen controls.	First numeric answer exact match; six-length macro (five-length 8k–128k macro for matched baseline tables).
LongBench	Six QA datasets; 1,150 examples total; 8 shards; seed 42.	32/64/128 tokens by dataset.	Official multi-reference token F1; equal-weight macro over six datasets.
BABILong	qa1/qa2/qa5; 0k–32k; 100 examples/cell; 4 shards; seed 42.	20 tokens.	Official <code>compare_answers</code> with <code>TASK_LABELS</code> .
LoCoMo	All 1,986 QA pairs from 10 conversations; 4 shards.	48 tokens for the main 8B comparison; 512 for the cross-scale audit.	GPT-4o judge on categories 1–4; official local abstention rule for category 5; full-set judge is the headline.

Table 23: Benchmark-level evaluation settings. Every main comparison uses `chat_template=False`. MemoryLLM is the external persistent-memory reference, evaluated with its released chat-tuned backbone.

Length	Full time	CoMem time	Speedup	Full mem.	CoMem mem.
8k	0.165 s	0.309 s	0.53×	19.92 GB	17.60 GB
16k	0.356 s	0.441 s	0.81×	24.55 GB	17.67 GB
32k	0.826 s	0.681 s	1.21×	33.82 GB	17.79 GB
64k	2.104 s	1.200 s	1.75×	52.34 GB	18.04 GB
128k	6.035 s	2.202 s	2.74×	89.39 GB	18.54 GB

Table 24: Same-platform Qwen3-8B pipeline efficiency with the same distilled rank-32 LoRA attached to both paths (one NVIDIA L20A, bf16/SDPA, median of three). “Full” processes the entire context; “CoMem” includes document Write, CPU retrieval, query Write, and upper-layer Read over the selected pack, but excludes one-time BM25 index construction and external-store I/O. Table 9 fixes the retrieved pack but compares unadapted $j=0$ with adapted $j=12$, isolating the operating-point trade-off. The select-first online path is reported separately in Table 5; it assumes a prewritten store and is not mixed with these full-pipeline component measurements.

8 KB/token (about 1 GiB at 128k), far more than text but $18\times$ less than full KV. Main timing cohorts keep the store in HBM; Table 28 separately measures placement beyond HBM, so its I/O microbenchmark is not added to model latency.

Paired quality–latency replay control. Table 26 uses a separate H20 harness. The two arms share the Qwen3-8B checkpoint, flagship LoRA SHA-256 and all 168 mounted modules, example IDs, decode parameters, and per-example retrieved chunk IDs, order, and pack tokens; only replay start differs. Across 15 RULER Cohort-B cells ($n=100$ each), full replay scores 99.19 and layer-12 replay scores 96.07. The paired 3.12-point gap has bootstrap 95% CI [2.36, 3.93] and exact McNemar $p=8.79\times 10^{-24}$ (83 full-only correct versus one layer-12-only correct). The gap is largest on distractor-heavy multikey retrieval.

For latency, each arm uses the same fixed 16k pack in three independent processes, with three warmups and 20 reads per process. Layer-12 replay is $1.403\times$ faster (931.9 to 664.4 ms), with all three process-level ratios between 1.402 and 1.404. Total-decode medians remain similar at roughly 2.76–2.86 s. A preceding component harness attributes the read reduction primarily to upper-transformer forward ($1.398\times$), with unchanged LM-head time. Retrieval, index construction, reusable store/query Write, and persistent I/O are outside this read-phase timing. The result is therefore a measured quality–latency trade-off, not

quality-preserving or end-to-end acceleration.

A.5 Training-seed robustness

We trained two additional $j=12$ rank-32 LoRA adapters on the same data budget and evaluated all three seeds, including the flagship, on identical cells and sample counts (RULER $n=50$, BABILong $n=100$). Across the 18 reported RULER and BABILong cells, the median cell-wise standard deviation is 1.34 points and the maximum is 4.36; the flagship falls within the observed range on the RULER headline cells and is, if anything, slightly conservative on the two BABILong cells that drive the maximum. The two added seeds use effective batch 3 rather than 8, so this remains a matched-data-budget robustness check with slightly different optimization noise rather than a fully controlled three-seed estimate.

A.6 Prompt and native-chat sensitivity

For Qwen3-8B, the GPT-4o LoCoMo judge is substantially less prompt-sensitive than token F1. CoMem scores 38.27 without chat templating and 37.76 with it; KV-Direct scores 34.59 and 38.22. In contrast, CoMem’s F1/substring accuracy changes from 9.15/23.36 to 19.51/28.65, while KV-Direct F1 changes from 9.02 to 40.06, showing that lexical metrics are strongly formatting-sensitive. We therefore retain the uniform no-chat judge comparison as the controlled headline.

The official MemoryLLM chat template does not improve the tested BABILong, RULER, or LoCoMo diagnostics; full results are included in the released artifacts. We therefore keep the uniform no-chat row as the external persistent-memory reference and do not mix prompt protocols within an average.

A.7 Cross-scale and MoE generality

Table 29 reports the completed no-chat adapter-free RULER sweep across six Qwen3 sizes. Performance is non-monotonic because the selected split, architecture, and task family vary with scale; we do not infer a scaling law. Table 30 evaluates a RULER-derived conservative integer split for each frozen backbone on all 1,986 LoCoMo questions. The no-chat score improves strongly overall (11.48→51.81) but is not strictly monotone. In particular, inspection of the 14B generations shows unusually long chain-of-thought trajectories without the chat template: reasoning consumes much of the generation budget before

(a) Quality at a 6,657-token/KV budget					
Method	Native A macro	Full-prompt 64k/128k		LoCoMo	
		single	multikey	tracking	F1/acc
CoMem+LoRA	97.05	99/100	91/93	99.0/95.8	9.15/23.36
SnapKV	72.33 [†]	100/100	94/91	80.0/84.8	9.21/22.05
PyramidKV	72.32 [†]	100/100	93/89	88.6/86.8	9.13/22.10

(b) Full-prefill systems cost on one H20					
Method	Len.	ms	GB	ms/tok	
SnapKV	8k	1,213	18.6	24.7	
	32k	6,342	25.7	24.6	
	128k	51,520	54.3	24.7	
PyramidKV	8k	1,203	18.6	24.7	
	32k	6,301	25.7	26.7	
	128k	51,520	54.3	26.8	

Table 25: Standard selected/compressed-KV baselines at CoMem’s retained budget. Panel (a) uses RULER Cohort A and full-set LoCoMo lexical scoring. CoMem reads its bounded pack from the full store without YaRN; the SnapKV/PyramidKV 64k/128k columns use YaRN factor 4 and see the full prompt. [†]Their native 15-cell macros include 64k/128k inputs left-truncated to the 40,960-token native window; within 8k–32k both score 99.96. Panel (b) times full-prompt prefill, compression, and 64-token decode (median of three); the final column is decode latency. These methods must prefill the full prompt before eviction. CoMem’s separate L20A cohort peaks at 18.54 GB at 128k (Table 24); hardware differs, so we report no cross-table latency ratio.

Replay start	RULER	Read (ms)	p10	p90
$j=0$	99.19	931.9	931.6	942.0
$j=12$	96.07	664.4	663.8	667.1
Trade-off	−3.12 pt	1.403× speedup		

Table 26: Paired quality–latency control with the same LoRA. Quality is the 15-cell RULER macro ($n=1,500$ paired examples); the 3.12-point loss has paired-bootstrap 95% CI [2.36, 3.93]. Latency uses a fixed 16k pack and reports medians across three independent processes (20 reads each). Retrieval, reusable Write, I/O, and total decode are excluded; total decode is similar across arms.

Replay pathway	RULER B	MK 8/16k	Reusable
Full replay from $j=0$	99.19	100.0	No
Continuous-prefix h_{12} oracle	99.19	100.0	No
Chunk-local cached h_{12}	96.07	92.5	Yes

Table 27: Write-side attribution with identical examples, packs, and LoRA. The continuous-prefix oracle runs layers 0–11 over the selected packed sequence for every query, then starts upper replay from that h_{12} ; it is bit-identical to full replay on all 1,500 Cohort-B examples. The deployable chunk-local path loses 3.12 points (95% CI [2.36, 3.93]; McNemar $p=8.79\times 10^{-24}$), and 7.50 points on the pooled 8k/16k multikey cells ($n=200$, CI [4.0, 11.5]). The oracle is an attribution upper bound, not a reusable cache.

Backend	Write GB/s	Fetch ms	H2D ms	Peak QPS	16M outcome
GPU resident	OOM	–	–	–	128 GiB > 95 GiB
CPU pinned	139.6	0.81	1.41	941	235 GB host
Local NVMe	2.45	13.42	0.91	264	2.18 GB host
Network/CEPH	1.66	79.20	1.20	42	2.19 GB host

Table 28: Measured persistent-store tiers at 16M tokens (128 GiB of bf16 h_{12}). Each query fetches the same 50.3 MB top-12 pack; medians use seven repetitions after two warmups, and QPS is the best of 1/4/16 concurrent clients. GPU residence fits 8M tokens (64.98 GB peak) but not 16M; off-GPU tiers retain store-size-independent transfer volume at increasing I/O latency. This microbenchmark times storage and transfer, not BM25 or model inference.

Model	j	single	multikey	VT	macro
0.6B	2	99.80	85.48	62.47	85.68
1.7B	3	99.08	48.08	44.20	66.81
4B	6	96.36	35.48	42.50	60.52
14B	13	95.28	34.44	13.10	52.91
30B-A3B	12	99.44	57.56	87.07	80.48
32B	27	100.00	93.60	59.47	88.18 [†]

Table 29: Completed adapter-free Qwen3-family RULER sweep. Values are means over five lengths for single and multikey (8k–128k), three for variable tracking (8k–32k), and all 13 cells for the macro. All cells use the same no-chat, chunk-512 protocol and $n=500$, except [†]32B ($n=25$). Scale is non-monotonic; architecture, selected split, and task family matter in addition to parameter count.

the final answer, plausibly explaining its dip below 8B. The chat template mitigates this behavior (28.80→39.58), while it slightly reduces 8B and 32B; we therefore treat the chat column as a generation-protocol sensitivity analysis rather than a second scaling curve. Table 31 adds Qwen3-30B-A3B, where Dense OOMs at 128k but CoMem runs at 63.0GB and scores 100 on RULER single needle.

A.7.1 Hunyuan Hy3: an 80-layer sparse MoE

We additionally port CoMem to Tencent Hunyuan Hy3, an 80-layer sparse Mixture-of-Experts model (192 experts, top-8 routing plus one shared expert; ≈ 597 GB in bf16) with a 262,144-token native window. The backbone is sharded across eight GPUs; Write and Read semantics are unchanged.

Exact partition through MoE routing. Reconstructing a contiguous forward by writing `layers[0:j]` and resuming `layers[j:80]` gives $\max|\Delta\text{logit}| = 0.0$ at $j \in \{0, 1, 40, 80\}$, including the Dense-to-MoE boundary. The re-

Model	Safe j	j/L	No chat	Chat
Qwen3-0.6B	2	0.07	11.48	20.49
Qwen3-1.7B	3	0.11	17.42	26.44
Qwen3-4B	9	0.25	24.07	32.02
Qwen3-8B	9	0.25	32.38	31.52
Qwen3-14B	13	0.325	28.80	39.58
Qwen3-32B	27	0.42	51.81	48.89

Table 30: **Frozen CoMem transfers across the Qwen3 family on LoCoMo.** Each integer split is a conservative readout-safe point selected from RULER, independently of LoCoMo; it is not the 50% readout-crash boundary in Table 6. Values are full-set GPT-4o Judge accuracy (%) over all 1,986 questions. Both protocols use iterative BM25 (top-12, hop width 4), 512-token chunks, a BOS sink, greedy decoding, and a 512-token generation cap; the chat column is a template-sensitivity control. The 14B no-chat dip reflects unusually long chain-of-thought trajectories that delay the final answer; the chat template mitigates this behavior. Scores improve strongly with scale but are not strictly monotone.

Length	Prefill speedup	Dense mem	CoMem mem
8k	1.4×	63.7 GB	63.0 GB
32k	8.4×	71.3 GB	63.0 GB
128k	Dense OOM	OOM	63.0 GB

Table 31: CoMem on **Qwen3-30B-A3B** (48-layer sparse MoE; prefill speedup = Dense/CoMem prefill time). CoMem peak memory is context-independent (63.0 GB across 8k/32k/128k) while the full-context Dense reference grows and *OOMs* at 128k on an NVIDIA H20. At 128k CoMem still runs and scores RULER niah_single 100 (Dense OOM→CoMem 100); its prefill stays < 0.7s at every length. The depth partition, retrieval, and constant read thus hold at sparse-MoE scale.

summed half re-executes every router and expert, so the partition preserves routing exactly on this self-test.

Split depth and self-distillation. A PG19 sweep places the best readout region near $j \approx 32$ –36. At $j = 32$, self-distillation reduces the 16k multiplicative LM tax from 1.496 to 1.078 and raises top-1 agreement from 0.773 to 0.871 (Table 32).

Bounded read through 256k. With the distilled adapter and BM25 top-8, the read stays at ≈ 4.3 –4.6k tokens from 16k through 256k. CoMem reaches 100/98 single/multikey recall at the 256k native ceiling with no OOM (Table 33).

$j=32$	ctx 8k		ctx 16k	
	gap	top1	gap	top1
zero-shot	1.378	0.823	1.496	0.773
distilled	1.071	0.895	1.078	0.871

Table 32: CoMem self-distillation on Hy3 at split $j=32$ (LoRA $r=32$, $\alpha=64$ on `layers[32:]`, teacher $j=0$ RAG recompute, 4000 steps of plain PG19 KL, no synthetic data). **gap** is the multiplicative LM tax (CoMem-readout perplexity / full-context perplexity; 1.0 is exact); **top1** is argmax agreement with the full forward, over matched 16-doc PG19 windows. The 16k LM tax falls from +49.6% to +7.8% and top1 rises 0.773 → 0.871, reproducing the 8B-backbone self-distillation effect on the 80-layer MoE.

Task	16k	32k	64k	128k	256k
niah_single_2	98	92	100	98	100
niah_multikey_1	100	94	100	100	98
read length (tok)	4.3k	4.6k	4.5k	4.4k	4.5k
context (tok)	16k	32k	65k	131k	261k

Table 33: CoMem on the Tencent Hunyuan **Hy3** backbone (hy_v3, 80-layer sparse MoE), RULER needle recall ($n=50$, official `string_match_all`), split $j=32$, BM25 top-8, self-distilled LoRA. The read length fed to the model stays ≈ 4.3 –4.6k tokens while the context grows $16\times$ (16k→256k); CoMem retains 100/98 recall at the backbone’s 256k native ceiling, with 0 out-of-memory failures. The context row reports the mean packed length of the RULER samples.

B Chat-Template-Free Statistical Verification

All statistics in this appendix were recomputed from the saved prediction shards on the shared result filesystem, without GPU reruns. The Qwen3-8B flagship uses the self-distilled $j=12$ adapter, chunk 512, top-12 retrieval, a BOS sink, and no chat template. The released adapter artifact is identified by the SHA-256 and configuration in Table 22. The main RULER directory uses the `iter_bm25` implementation; the saved configuration records `rounds=0` and an observed read of approximately 6.5k tokens. Every reported RULER cell contains eight of eight shards, 100 examples, no empty prediction, and zero mismatch between the stored score and a fresh official-kernel recomputation. BABILong likewise contains 100 valid examples in every cell.

B.1 RULER cohort definitions

Cohort A is the all-method table (Appendix Table 16): it substitutes `niah_single_2`, keeps the same multikey/tracking tasks and $n=100$, and gives CoMem 97.05. Cohort B above was freshly sampled with `niah_single_3` because that is the exact task available for both YaRN arms and $j=0$; it gives 96.07. Tables mark every RULER aggregate with *A* or *B*, never subtract across them, and exclude legacy 256k cells. Table 3 contains only Cohort B.

Task	8k	16k	32k	64k	128k
<code>niah_single_3</code>	100.0	91.0	97.0	98.0	98.0
<code>niah_multikey_1</code>	94.0	91.0	99.0	90.0	93.0
<code>variable_tracking</code>	96.2	98.0	98.2	98.6	99.0

Table 34: RULER **Cohort B**, used for the matched length-extension and $j=0$ comparisons. The $n=100$ macro is $1441.0/15 = 96.0667$; the exact single-needle variant is `niah_single_3`.

B.2 LoCoMo judge protocol and denominator

Prediction shards for all methods and the adapter-free CoMem control deduplicate to all 1,986 LoCoMo items. GPT-4o judges the 1,540 answerable items in categories 1–4. Category 5 contains 446 adversarial abstention items and is scored locally: an answer is correct iff it is empty or matches the refusal rule. Thus the canonical full-set judge is over all 1,986 items, not only the API-judged subset.

Method	Cat. 1	Cat. 2	Cat. 3	Cat. 4	Cat. 5	Overall
CoMem + LoRA	26.95	19.00	30.21	69.32	2.47	38.27
CoMem frozen ($j=9$)	18.79	13.40	27.08	53.75	1.12	29.15
KV-Direct	24.11	18.69	25.00	62.19	2.69	34.59
StreamingLLM	22.70	11.21	23.96	42.21	6.95	25.63
InfLLM	18.44	14.33	25.00	33.89	7.62	22.21
MemoryLLM	15.60	8.72	15.63	27.47	0.45	16.11

Table 35: LoCoMo GPT-4o judge by category. Categories 1–4 contain 282/321/96/841 judged items; Category 5 contains 446 locally scored abstention items. Overall aggregates all 1,986 examples.

The judge uses an OpenAI-compatible chat-completions endpoint with model name `gpt-4o`, `seed=1`, and no client-set temperature or top- p . The endpoint does not expose a dated model snapshot. Requests are retried four times with exponential backoff; unparsable responses and API failures are conservatively scored wrong. The prompt presents the question, gold answer, and prediction; it accepts any semantic match (including equivalent date phrasing), rejects contradictions, omissions, empty answers, and refusals, and requires exactly `CORRECT` or `WRONG`. The verbatim template is released with the evaluation script. Gold alternatives are joined with “OR.” A reply must begin with `CORRECT` or `WRONG`; a fallback substring vote is used before defaulting to wrong.

On the common GPT-4o-judged items ($n=1540$), CoMem scores 48.64 and KV-Direct 43.83. A paired per-item bootstrap of their difference (10,000 resamples, seed 1234)

gives +4.81 points with 95% CI [2.34, 7.34]. Because the questions are nested within only 10 conversations, this interval is not our sole inferential check. A paired conversation-cluster bootstrap, resampling the 10 conversations rather than individual questions (same 10,000 resamples, seed 1234), gives the same +4.81 point estimate with 95% CI [1.27, 8.32] — wider than the per-item interval, as expected with only ten clusters; 8 of the 10 observed conversation-level differences favor CoMem. We do not interpret the bootstrap tail fraction as a calibrated hypothesis-test p value. We report the latter as a dependence-aware robustness check rather than claiming that 1,540 questions are independent. The canonical full-set point scores are 38.27 and 34.59, respectively.

B.3 Independent-judge audit

We reran the binary prompt with DeepSeek-V3 on a stratified 200-item subset shared by CoMem and KV-Direct. Agreement with GPT-4o is 0.81 ($\kappa = 0.626$). Both judges preserve the ordering: GPT-4o scores 36.5/32.5 (+4.0) and DeepSeek-V3 56.0/49.0 (+7.0) for CoMem/KV-Direct. Absolute calibration differs, but the positive gap is judge-robust.

B.4 Uncertainty and aggregation checks

Per-method, 1,000-resample item-bootstrap intervals (seed 1234) are [36.20, 40.58] for CoMem’s full-set LoCoMo judge, [32.48, 36.71] for KV-Direct’s, [46.04, 50.98] and [41.23, 46.30] for their category-1–4 judge scores, and [8.60, 9.72] and [8.51, 9.61] for their F1 scores. These unpaired, item-level intervals are descriptive; the paired cluster analysis above is the appropriate dependence-aware comparison.

We do not reuse cell-bootstrap intervals from the superseded RULER cohort or legacy BABELLong predictions. The separate six-length LongEval aggregate is 72.83 over 600 examples (95% CI [69.33, 76.33]); Table 2 uses the predefined five-length 8k–128k mean of 69.0.